Universidad del Rosario

Prueba corta — Segundo corte

Estudiante	Fecha
David Santiago Arteaga Pinilla	23 / 10 / 2026

Duración	Entrega	Modalidad
45 min de desarrollo + 10 min de entrega	Repositorio del equipo + evidencias	Individual, en computador

Instrucciones generales

- Trabajo individual, en su computador. Permitido: la documentación oficial de C++ y de Python, repositorio personal. No permitido: apuntes compartidos, mensajería, repositorios de terceros ni asistentes de inteligencia artificial de ningún tipo.
- Descargue la carpeta de la prueba. Modifique únicamente los archivos recursion.py, busqueda.cpp, historial.py y zonas.cpp.
- Los archivos pruebas_e1.py, pruebas_e2.cpp, pruebas_e3.py y pruebas_e4.cpp NO se pueden modificar. Alterarlos se considera fraude académico según la sección 13 de la guía de asignatura.
- Ejecute las pruebas con bash verificar.sh en Linux o macOS, o con verificar.bat en Windows. Para un solo ejercicio, agregue su nombre: bash verificar.sh E2.
- Cuando un ejercicio queda resuelto, las pruebas imprimen su código de cierre. Ese código no está escrito en ningún archivo: se calcula a partir de los resultados correctos. Cópielo en la hoja de respuestas de la última página.
- No se evalúa la velocidad para escribir código, sino que la solución sea correcta y que usted pueda explicarla.

Entrega en el repositorio — obligatoria

Ojo: Un ejercicio sin código en el repositorio no se califica, aunque las pruebas hayan pasado en su pantalla.

Paso	Qué debe hacer
1. Carpeta	Suba E1, E2, E3 y E4 con su código a prueba-corta-c2 del repositorio individual dentro de segundo corte.
2. Commit	Mensaje exacto: prueba corta c2- <descripción> - - <apellido> - <nombre>
3. Evidencias	Cree la subcarpeta evidencias/ con una captura por ejercicio donde se vea la salida completa de las pruebas y el código de cierre.
4. Push	Haga push ANTES de que termine el tiempo. El commit debe quedar registrado dentro del horario de la prueba; la hora del commit es la que cuenta.
5. Sustentación	Antes de retirarse, muestre a la profesora la salida de las pruebas en su pantalla y responda una pregunta sobre el código que escribió.

Universidad del Rosario

Prueba corta — Segundo corte

Distribución del puntaje

Ejercicio	Tema	Lenguaje	Pruebas y repositorio	Justificación escrita	Total
1	Recursión	Python	20	5	25
2	Búsqueda binaria	C++	20	5	25
3	Pilas	Python	20	5	25
4	Mapas y conjuntos	C++	20	5	25
		TOTAL	80	20	100

Universidad del Rosario

Prueba corta — Segundo corte

EJERCICIO 1 · Recursión · Python · 25 puntos

Archivo que debe modificar: `E1/recursion.py` · Pruebas: `python3 pruebas_e1.py`

Sistema TurnoJusto — cálculo de rutas de atención. El archivo contiene dos funciones.

(a) Corrija el defecto de suma_impares (10 puntos)

La función debe devolver la suma de todos los números impares desde 1 hasta n, incluido. Se reporta que con ciertas entradas el programa se cae. El defecto está en el caso base.

Código entregado — tiene un defecto

```
def suma_impares(n):
    if n == 1:
        return 1
    if n % 2 == 0:
        return suma_impares (n - 1)
    return n + suma_impares (n - 1)
```

Resultados esperados: `suma_impares (5) → 9  suma_impares (1) → 1  suma_impares (0) → 0  suma_impares (10) → 25`

(b) Implemente contar_caminos (10 puntos)

Devuelve cuántos caminos distintos hay desde la casilla superior izquierda hasta la inferior derecha de una cuadrícula de f filas por c columnas, avanzando solo a la DERECHA o hacia ABAJO. Debe ser recursiva: no use ciclos ni listas.

Resultados esperados: `contar_caminos (3,3) → 6  contar_caminos (2,4) → 4  contar_caminos (1,9) → 1  contar_caminos(4,4) → 20`

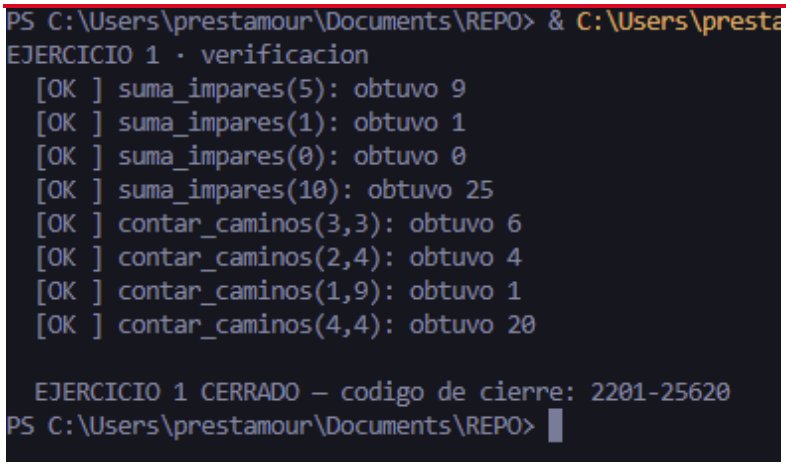
(c) Justificación escrita (5 puntos)

¿Con qué entrada concreta se caía la función original y por qué? Nombre el error que produce Python en ese caso.

El código en concreto se caía con la entrada de suma_impares(0) Puesto que el 0 no estaba definido como el caso base; haciendo que nunca llegara al mismo y provocando un RecursionError provocado por nunca alcanzar el return de salida.

Universidad del Rosario

Prueba corta — Segundo corte



Universidad del Rosario

Prueba corta — Segundo corte

EJERCICIO 2 · Búsqueda binaria · C++ · 25 puntos

Archivo que debe modificar: `E2/busqueda.cpp`

Pruebas: `g++ -std=c++17 -g -o pruebas_e2 busqueda.cpp pruebas_e2.cpp &&. /pruebas_e2`

Sistema RedAcopio — localización de códigos de material. El arreglo está ordenado de forma no decreciente. Se reportan dos fallas distintas:

- Devuelve −1 para códigos que sí están en el arreglo.
- Con algunos valores lee una posición que no pertenece al arreglo. Las pruebas lo detectan con un centinela: si la función devuelve el índice 10 sobre un arreglo de 10 elementos (índices 0 a 9), está leyendo fuera de rango.

Código entregado — tiene dos defectos

```
int buscarCodigo (int v [], int n, int x) {
    int izq = 0, der = n;                       // <-- revise
    while (izq <= der) {                        // <-- revise
        int medio = (izq + der) / 2;
        if (v[medio] == x) return medio;
        if (v[medio] < x) der = medio - 1;      // <-- revise
        else              izq = medio + 1;      // <-- revise
    }
    return -1;
}
```

(a) Corrija los dos defectos (20 puntos)

Las diez pruebas deben quedar en OK, incluida la del centinela. Este archivo no tiene main: el main está en pruebas_e2.cpp y se compilan juntos.

(b) Justificación escrita (5 puntos)

Cuando el programa buscaba el valor 250, ¿qué posición estaba leyendo y por qué eso es un error, aunque el resultado devuelto pareciera correcto?

El programa solo tenia permitido leer dentro del rango de n (en código seria el rango de 0-9) de manera que no se debía tomar en cuenta el 250 a pesar de que estuviera no podía acceder a su posición ya que nunca entraba a buscar más allá de n; de manera que si se llegaba a entregar un vector de mayor tamaño leería hasta el n indicado que era 10 en este caso y pues omitía el 250 por eso entrega -1 ya que no lo encuentra.

Universidad del Rosario

Prueba corta — Segundo corte

```
PS C:\Users\prestamour\Documents\REPO\Trabajos-Algoritmos\SegundoCorte\Parcial2\prueba_corta_corte2\E2> ./pruebas_e2
EJERCICIO 2 ┬À verificacion
  [OK ] buscar 163 (interior): obtuvo 6
  [OK ] buscar 104 (primero): obtuvo 0
  [OK ] buscar 197 (ultimo): obtuvo 9
  [OK ] buscar 138: obtuvo 3
  [OK ] buscar 150 (ausente, medio): obtuvo -1
  [OK ] buscar 100 (menor que todos): obtuvo -1
  [OK ] no lee fuera del arreglo (buscar 250): devolvio -1
  [OK ] buscar 250 (mayor que todos): obtuvo -1
  [OK ] arreglo de 1: buscar 42: obtuvo 0
  [OK ] arreglo de 1: buscar 7: obtuvo -1

  EJERCICIO 2 CERRADO ÔÇö codigo de cierre: 2202-69318
```

Universidad del Rosario

Prueba corta — Segundo corte

EJERCICIO 3 · Pilas · Python · 25 puntos

Archivo que debe modificar: `E3/historial.py` · Pruebas: `python3 pruebas_e3.py`

Sistema PrestaLab — historial de acciones del operador. Cada cadena distinta de "DESHACER" se agrega al historial; "DESHACER" retira la última acción registrada y, si el historial está vacío, se ignora sin producir error.

(a) Corrija el defecto de historial_final (10 puntos)

Se reporta que el programa se cae cuando llega un "DESHACER" y el historial está vacío.

Código entregado — tiene un defecto

```
def historial_final(acciones):
    pila = []
    for a in acciones:
        if a == "DESHACER":
            pila.pop ()                 # <-- revise
        else:
            pila. append(a)
    return pila
```

(b) Implemente profundidad_maxima (10 puntos)

Devuelve el mayor número de acciones que el historial llegó a tener en algún momento del recorrido.

Ejemplo

```
profundidad_maxima (["a", "b", "DESHACER", "c"]) -> 2
# el historial llego a tener 2 elementos: a y b
```

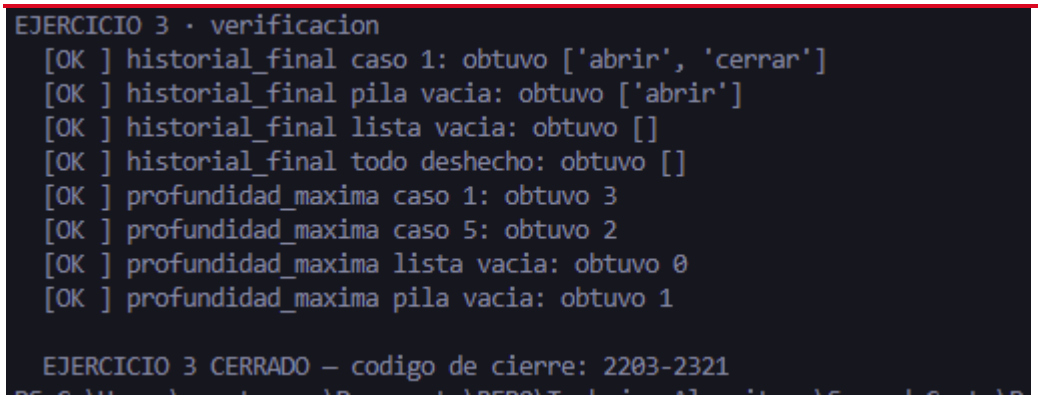
(c) Justificación escrita (5 puntos)

¿Por qué una pila es la estructura adecuada para este problema? Indique la complejidad de agregar y de retirar un elemento.

Necesitamos una pila puesto que para la funcionalidad que queremos aplicar que es la de corregir un error mediante la funcionalidad de deshacer. La cual necesita saber la última acción realizada y de la misma manera hacer la corrección es decir el ultimo que entra es el primero que sale. Me gusta compararlo como cuando lavamos platos, en donde el que se encuentra más arriba en la pila de platos es el que normalmente vamos a lavar. Por otro lado, la complejidad de encolar y desencolar es de O(1) ya que simplemente accedemos al ultimo agregado y eliminarlo.

Universidad del Rosario

Prueba corta — Segundo corte



Universidad del Rosario

Prueba corta — Segundo corte

EJERCICIO 4 · Mapas y conjuntos · C++ · 25 puntos

Archivo que debe modificar: `E4/zonas.cpp`

Pruebas: `g++ -std=c++17 -g -o pruebas_e4 zonas.cpp pruebas_e4.cpp &&. /pruebas_e4`

Sistema RedAcopio — consolidado de recolección por zona. Una misma zona puede aparecer en varios registros.

(a) Implemente zonaMayor (20 puntos)

Firma — no la cambie

```
string zonaMayor (const vector<pair<string, int>>& reg,
                  map<string, int>& totales,
                  set<string>& repetidas);
```

La función debe hacer tres cosas:

- Llenar totales con los kilos acumulados por zona.
- Llenar repetidas con las zonas que aparecen en más de un registro.
- Devolver el nombre de la zona con mayor total acumulado.

Caso borde que las pruebas verifican: si `reg` está vacío, devuelva la cadena vacía `""` y no agregue nada a los contenedores.

Datos de la prueba

```
reg = {{"norte",40}, {"sur",25}, {"norte",15}, {"centro",60},
       {"sur",30}, {"oriente",18}, {"centro",5}}
```

(b) Justificación escrita (5 puntos)

¿Por qué conviene un map y no un vector de parejas para acumular? Indique la complejidad de una inserción o consulta en std::map y en std::unordered_map, y a qué se debe la diferencia.

Se utiliza un map y no un vector de parejas por su complejidad. Puesto que la de un vector de parejas necesita recorrerlo cada vez que se va a hacer el cálculo; resultando en una complejidad de O(n^2).En cambio, utilizando el map y unordered_map se hace de una vez la búsqueda del dato que estamos buscando; resultando en una complejidad de O(n).

Universidad del Rosario

Prueba corta — Segundo corte

```
EJERCICIO 4 ┬A verificacion
  [OK ] zona con mayor total: obtuvo "centro"
  [OK ] cantidad de zonas: obtuvo 4
  [OK ] total de norte: obtuvo 55
  [OK ] total de sur: obtuvo 55
  [OK ] total de centro: obtuvo 65
  [OK ] total de oriente: obtuvo 18
  [OK ] cantidad de zonas repetidas: obtuvo 3
  [OK ] oriente NO esta en repetidas: obtuvo 0
  [OK ] centro SI esta en repetidas: obtuvo 1
  [OK ] lista vacia devuelve "": obtuvo ""
  [OK ] lista vacia no llena totales: obtuvo 0

  EJERCICIO 4 CERRADO ÔÇö codigo de cierre: 2204-6543
```

Universidad del Rosario

Prueba corta — Segundo corte

HOJA DE RESPUESTAS — entregar firmada

Diligencie esta página y entréguela a la profesora antes de retirarse del salón.

Estudiante	Fecha
David Santiago Arteaga Pinilla	23/10/2026

1. Códigos de cierre

Transcriba exactamente lo que imprimieron las pruebas. Si un ejercicio quedó abierto, escriba ABIERTO.

Ejercicio	Tema	Código de cierre
1	Recursión (Python)	2201- 25620
2	Búsqueda binaria (C++)	2202-69318
3	Pilas (Python)	2203-2321
4	Mapas y conjuntos (C++)	2204- 6543

2. Entrega en el repositorio

Dato	Valor
URL del repositorio	santii85233/Trabajos-Algoritmos
Rama	main
Carpeta	SegundoCorte/Parcial2
Hash del último commit (7 caracteres)	7e6f91a
Hora del push	8:38 a.m
Capturas subidas a evidencias/	☐ E1 ☐ E2 ☐ E3 ☐ E4